How can I remove a large file from my commit history?



deployhq.com/git/faqs/removing-large-files-from-git-history



If you've committed a large file to your repository that takes up a significant amount of disk space, simply removing it in a later commit won't actually help. Git stores the full history of every file, so even after you delete a file from your working directory, it remains in Git's history in case you want to restore it.

To truly remove the file, you need to rewrite the repository's history.

Using `git filter-repo` (recommended)

`git filter-repo` is the modern replacement for the older `filter-branch` command. It's faster, safer, and the approach recommended by the Git project itself.

First, install it:

```
# macOS
brew install git-filter-repo

# pip (any platform)
pip3 install git-filter-repo
```

Then remove the file from your entire history:

```
git filter-repo --invert-paths --path path/to/ceo.jpg
```

That's it. The file is gone from every commit. If you need to remove multiple files or a directory:

```
git filter-repo --invert-paths --path assets/videos/ --path old-backup.sql
```

Note: `git filter-repo` requires a fresh clone by default. If you're working on an existing checkout, add `--force` --- but make sure you have a backup first.

Using BFG Repo-Cleaner

BFG Repo-Cleaner is another popular option, especially for removing large files by size. It's a Java tool that's simpler than filter-repo for bulk cleanup:

```
# Remove all files over 50MB from history
java -jar bfg.jar --strip-blobs-bigger-than 50M my-repo.git

# Remove a specific file by name
java -jar bfg.jar --delete-files ceo.jpg my-repo.git
```

After running BFG, clean up the repository:

```
cd my-repo.git
git reflog expire --expire=now --all
git gc --prune=now --aggressive
```

Using `git filter-branch` (deprecated fallback)

The older `filter-branch` command still works but is deprecated and significantly slower on large repositories. Use it only if `filter-repo` and BFG are unavailable:

```
git filter-branch --force --index-filter \
  'git rm --cached --ignore-unmatch path/to/ceo.jpg' \
  --prune-empty --tag-name-filter cat -- --all
```

Force push to your remotes

After rewriting history with any of the tools above, you'll need to force push to update the remote:

```
git push origin --force --all
```

Force pushing rewrites commits that other developers may have based work on. If you're on a team, coordinate with them before running this --- or read up on the consequences of force pushing.

The space won't be reclaimed on the remote host immediately. You'll need to wait for garbage collection to run before the size is recalculated.

Preventing large files in the future

Once you've cleaned up, take steps to avoid the same problem again:

Update your `.gitignore` to exclude files that shouldn't be tracked. We've written a beginner's guide to `.gitignore` files that covers the basics.

Use Git LFS for files that need to be in the repo but are too large for regular Git. LFS replaces large files with lightweight pointers while storing the actual content on a separate server. See our guide on managing Git LFS with DeployHQ for setup instructions.

Related

- Ignoring files with `.gitignore`
- Managing Git LFS and DeployHQ
- `git push` command reference

DeployHQ works with repositories of any size. If you're using Git LFS, DeployHQ supports it out of the box --- get started free.